

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

Ch4: What is Pandas & Why Pandas?

Ch5: What is Matplotlib & Why Matplotlib?

Ch6: What is Sklearn & Why Sklearn?

Ch7: What is Data Preprocessing & Why Data Preprocessing?

왜 Python OOP (Object-Oriented Programming)
은 필요한가?

==

왜 우리는 Data Type(Class)를 만들어야 하나?

Built-in Data Type vs. User-defined Data Type

- **Python Basic Data Types (기본 데이터 타입)**

- `int`
- `float`
- `Bool`

- **Python Collection Data Types**

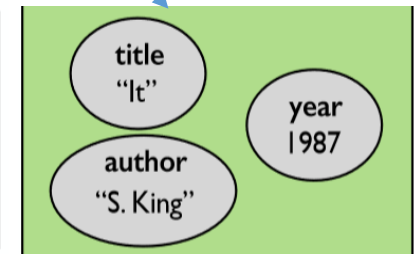
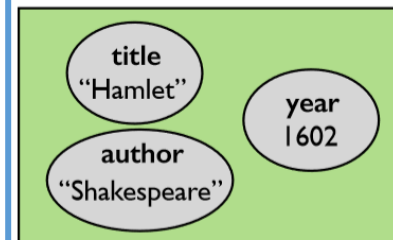
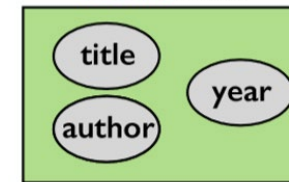
(컨테이너 데이터 타입)

- `str`
- `list`
- `tuple`
- `set`
- `dict`

Books in a library

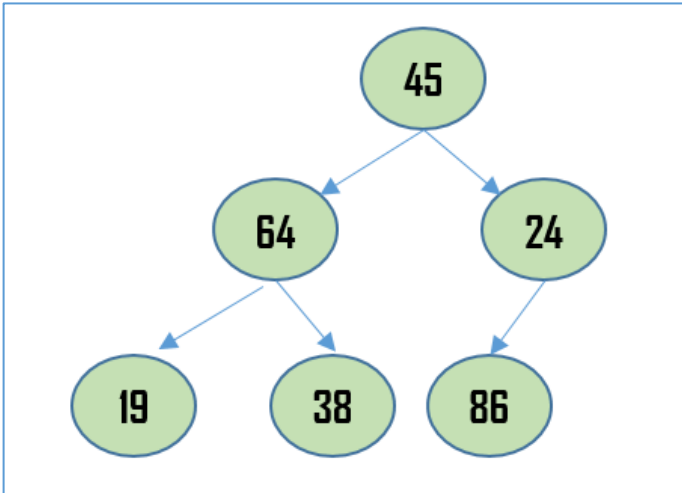


Book class



Python Object Oriented Programming (OOP)

Tree 구조에 데이터를 저장하고 싶다면 [1/2]



- Insert a new node
- Delete an existing node
- Search a location of a particular value
- Find min/max among values in the tree

• ...

List로 구현

[45, 64, 24, 19, 38, None, 86]

Level 1 Level 2 Level 3

Nested List 구현

- [45, left, right]
- [45, [64, left, right], [24, left, []]]
- [45, [64, [19, [], []], [38, [], []]], [24, [86, [], []], []]]

Tree 구조에 데이터를 저장하고 싶다면 [2/2]

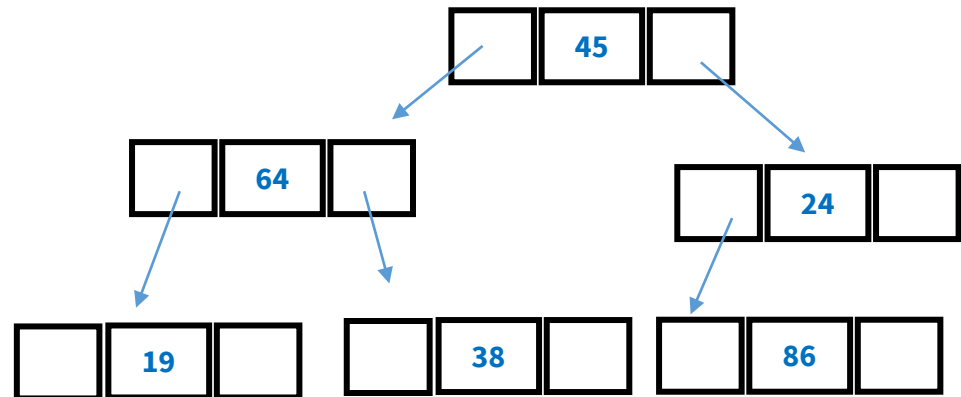
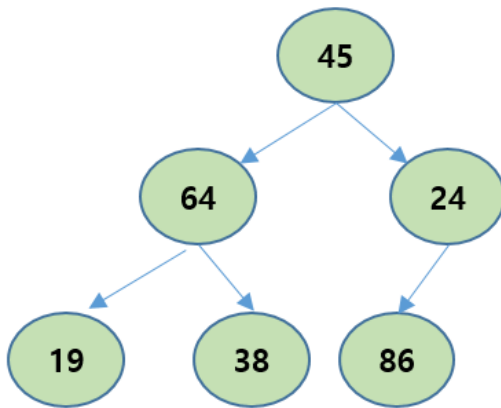
Tree Structure를 위한 Data Type이 있으면 좋겠다!

User-defined Type (class) → BinaryTreeNode Class

Left
Pointer

Node
Value

Right
Pointer



Motivating Example of OO: 도서관 책정보 정리

[1/3]

- Suppose we want to keep track of the books in you
- For each book, you want to store: `title`, `author`, `year_published`
- **Example**
 - Book1: “The Catcher in the Rye” , “J. D. Sallinger” , 1951
 - Book2: “The Brothers Karamazov” , “F. Dostoevsky” , 1880

- **Option1: Using Only Variables in Python**

```
book1Title = “The Catcher in the Rye”
```

```
book1Author = “J. D. Sallinger”
```

```
book1Year = 1951
```

```
book2Title = “The Brothers Karamazov”
```

```
book2Author = “F. Dostoevsky”
```

```
book2Year = 1880;
```



Motivating Example of OO: 도서관 책정보 정리 [2/3]

- Example

- Book1: “The Catcher in the Rye”, “J. D. Sallinger”, 1951
- Book2: “The Brothers Karamazov”, “F. Dostoevsky”, 1880

- Option 2: Using List in Python

```
book1 = [“The Catcher in the Rye”, “J.D. Sallinger”, 1951]
```

```
book2 = list( )
```

```
book2.append(“The Brothers Karamazov”)
```

```
book2.append(“F. Dostoevsky”)
```

```
book2.append(1880)
```



앞page에서 variable만 가지고 표현할때보다는 많이 발전된 상황이지만,

여전히 List로 표현하는 variable을 계속 만들어야 하고, User가 0번째 data가 책이름, 1번째 data가 저자. 2번째 data가 발행년도라는것을 계속 챙겨야 한다!

Motivating Example of OO: 도서관 책정보 정리 [3/3]

- Example

- Book1: “The Catcher in the Rye” , “J. D. Sallinger” , 1951
- Book2: “The Brothers Karamazov” , “F. Dostoevsky” , 1880

- Option 3: Using Dictionary in Python

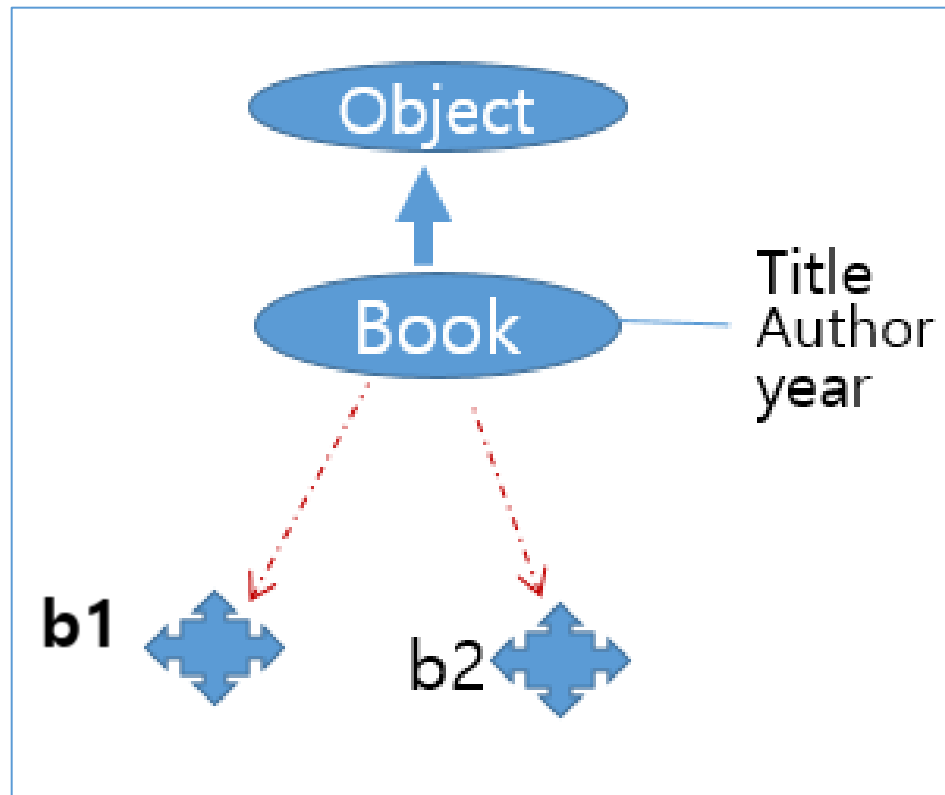
```
book1 = { “title”: “The Catcher in the Rye”,  
          “author”: “J.D. Sallinger”,  
          “year”: 1951 }
```

```
book2 = dict(  
book2[“title”] = “The Brothers Karamazov”  
book2[“author”] = “F. Dostoevsky”  
book2[“year”] = 1880
```

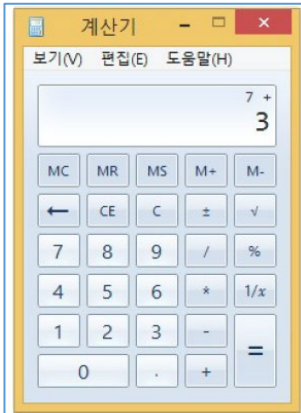


앞page에서 List를 가지고 표현할때보다는 많이 발전된 상황이지만,
여전히 Dictionary를 표현하는 variable을 계속 만들어야 한다

Book Data Type (= Book Class) 가 있으면
Coding이 수월하겠네!!



Motivating Example of OOP : 계산기 구현 [1/3]



- 만약 한 프로그램에서 `adder()`를 구현했다고 하자
- 그런데 2개의 계산기가 필요하다면 어떻게 해야 할까?
- 각각의 계산기는 각각의 결과값을 유지해야 하므로 `adder()` 1개로는 결과값을 따로 유지할 수 없다

Using Python Functions Only

```
result = 0
```

```
def adder(num):
    global result
    result = result + num
    return result
```

```
print(adder(3))
print(adder(4))
```

```
3
7
```

Using Python Functions Only

```
result1 = 0
result2 = 0
```

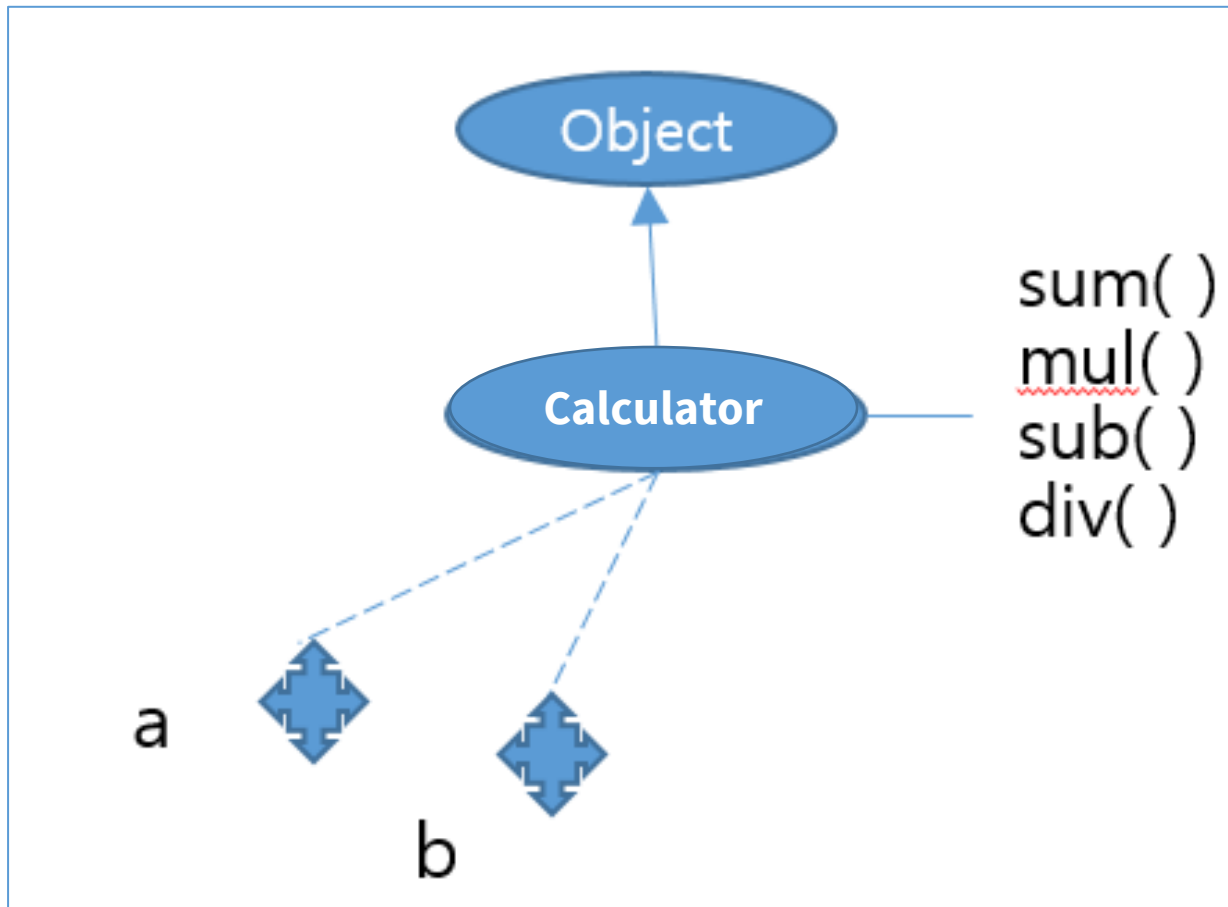
```
def adder1(num):
    global result1
    result1 = result1 + num
    return result1
```

```
def adder2(num):
    global result2
    result2 = result2 + num
    return result2
```

```
print(adder1(3))
print(adder1(4))
print(adder2(3))
print(adder2(7))
```

```
3
7
3
10
```

Calculator Data Type (= Calculator Class) 가
있으면 Coding이 수월하겠네!!



Python OOP 주요개념들

Python Class 생성 예

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        print(self.name)
```

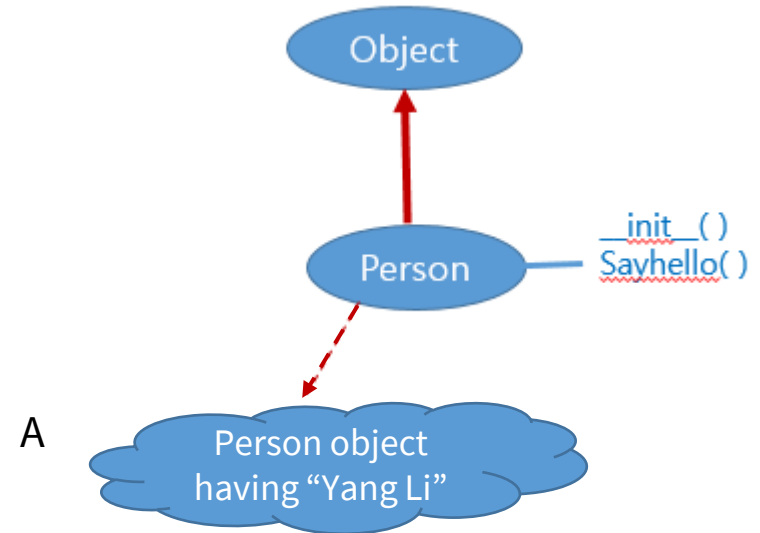
```
    def Sayhello(self):
```

```
        print('Hello, my name is', self.name)
```

```
A = Person('HJ Kim')  
'HJ Kim'
```

```
A.Name  
'HJ Kim'
```

```
A.SayHello()  
'Hello, my name is HJ Kim'
```



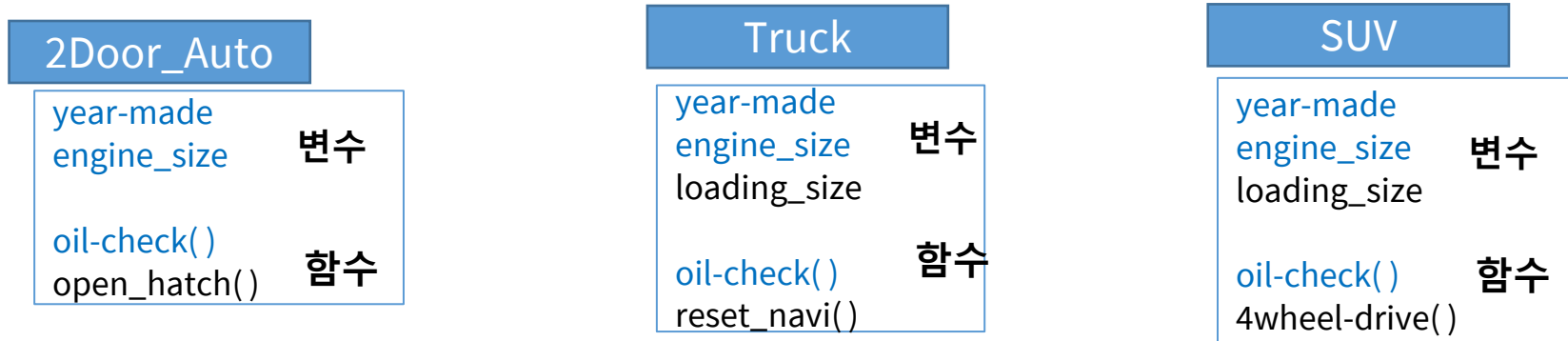
용어

- class definition
 - function definition (= method definition)
 - `__init__()`: constructor function
 - variable definition (= attribute definition)

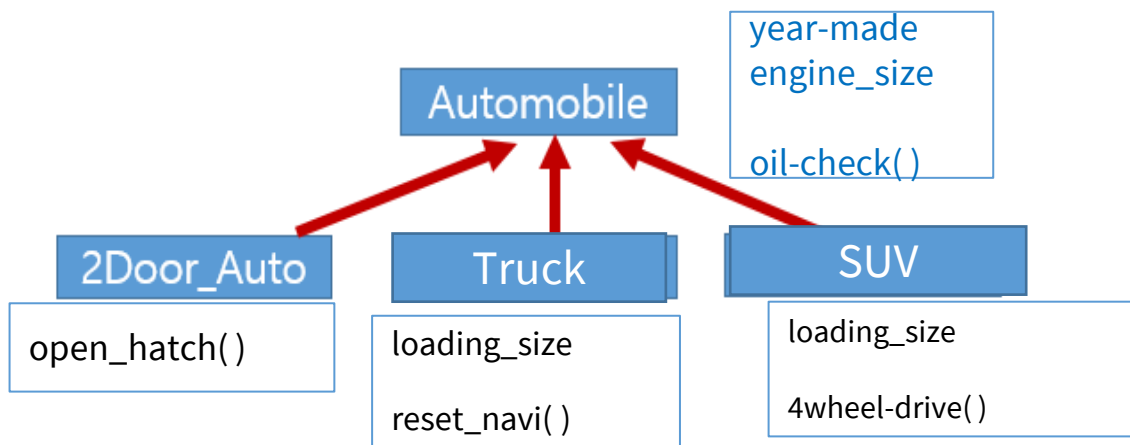
OOP의 상속 개념

[1/2]

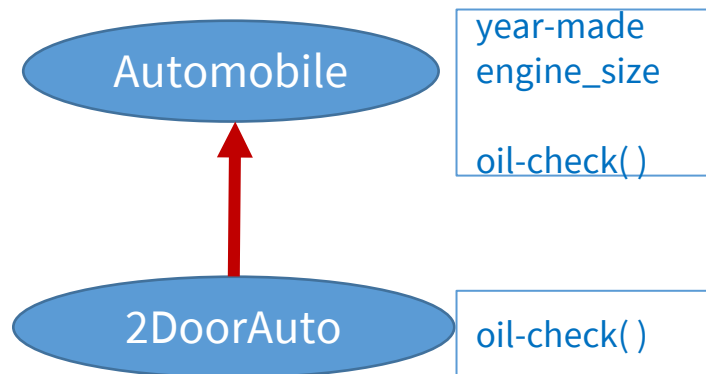
- 상황: 2Door_Auto, 4Door_Auto, Sedan 등등의 class들을 대표하는 Class를 두고 싶다



- 해결책: 2Door_Auto, Truck, SUV의 공통으로 들어 있는 Instance Variable 과 Method로 구성된 Automobile Class 를 Superclass로 위치하게 한다 → Code Factorization



- **Superclass (부모 클래스)로부터 새로운 subclass(자식 클래스)를 파생(생성)**
 - Subclass는 Superclass로부터 **변수와 함수를 상속**
 - Subclass 는 동일한 이름의 변수와 함수를 **직접 정의함**으로써 Superclass의 변수와 함수를 오버라이드(재정의) 할수 있음



동의어

Variable: Attribute = Instance Variable = Data Member = Property = Representation

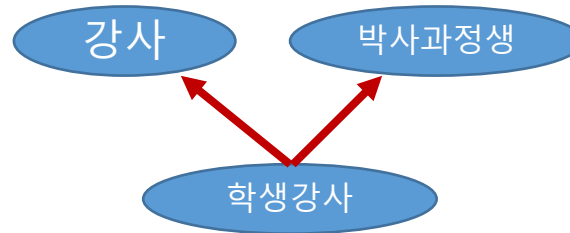
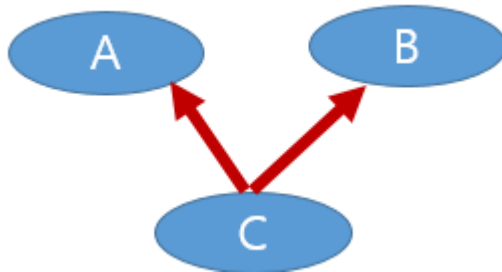
Function: Method = Operation = Member Function = Procedure = Behavior

SuperClass: ParentClass = BaseClass

SubClass: ChildClass = DerivedClass

- 다중상속의 개념 (Multiple Inheritance)
- Subclass는 여러 개의 superclass를 가질수 있다. (도출될수 있다)

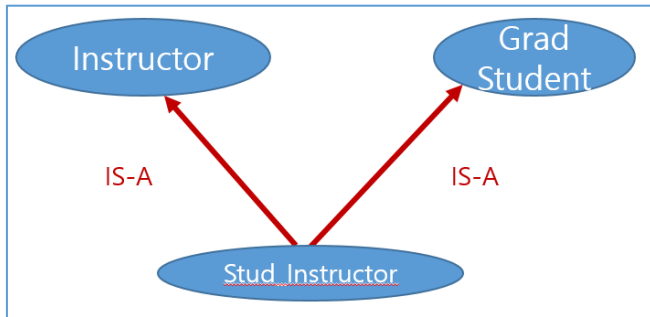
```
class SubClassName (ParentClass1, ParentClass2, ...):  
    'Optional class documentation string'  
    class_suite
```



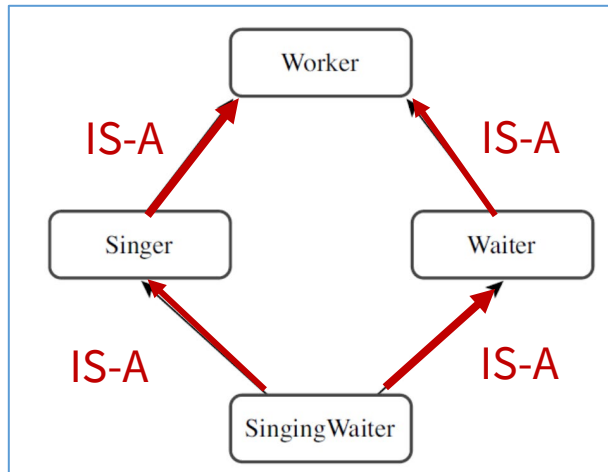
```
class A:      # define your class A  
.....  
class B:      # define your calss B  
.....  
class C(A, B): # subclass of A and B
```

```
class 강사():  
.....  
class 박사과정생():  
.....  
class 학생강사(강사, 박사과정생):  
.....
```

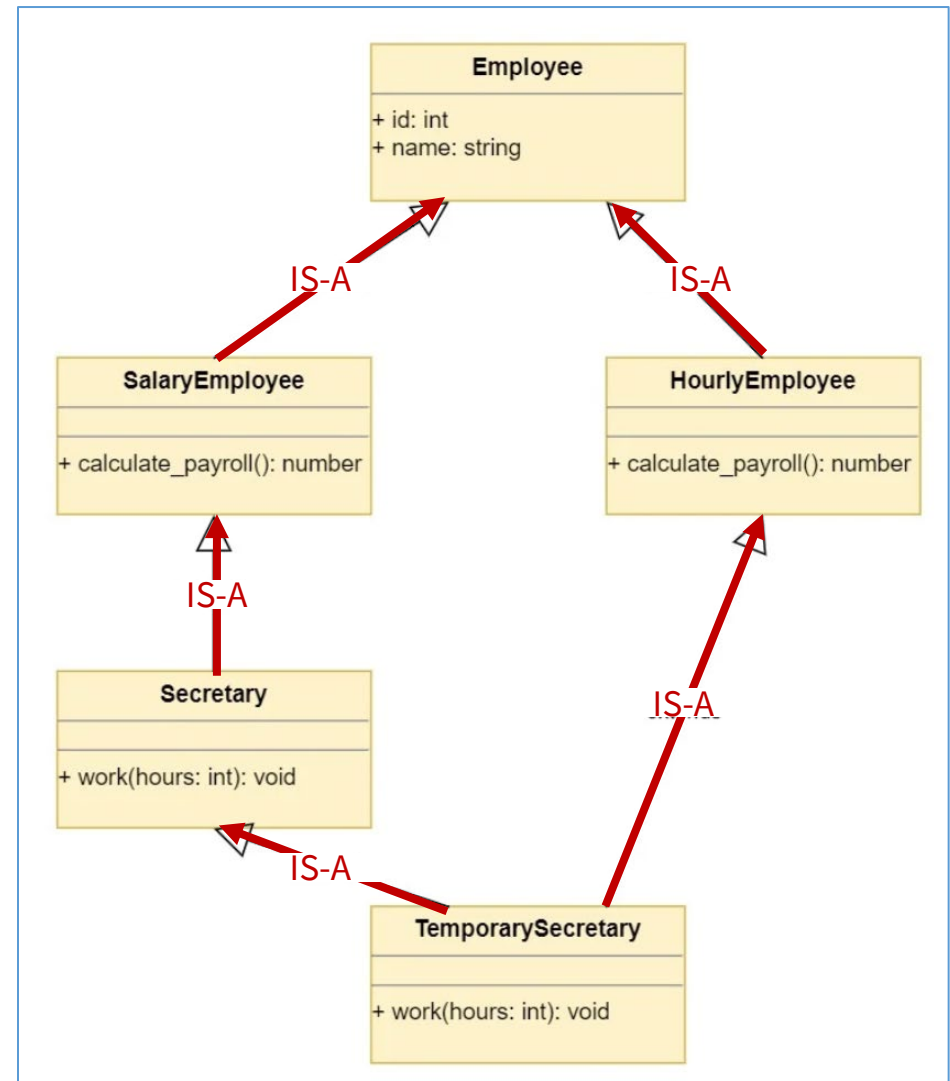
Multiple Inheritance의 예



대학원생_강사



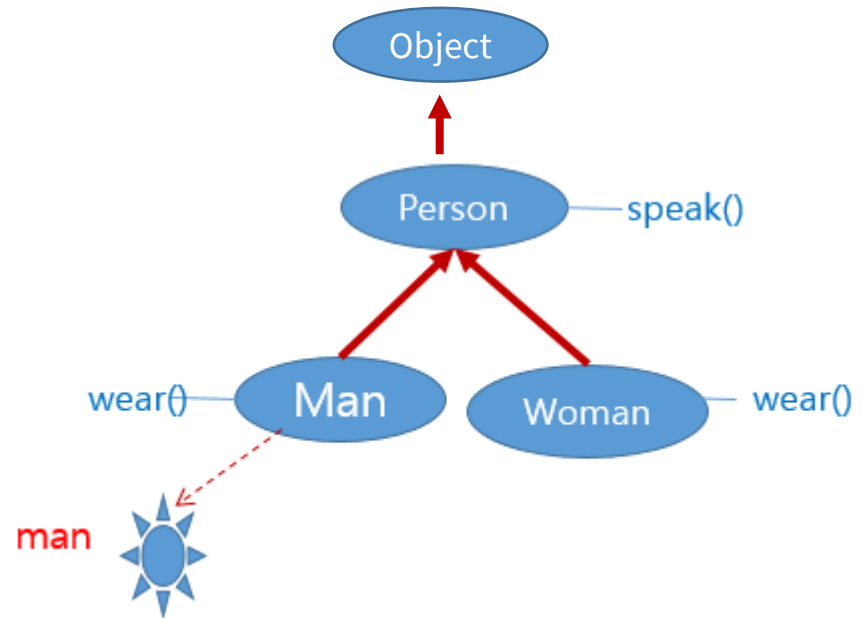
가수_웨이터



시간제_비서

Superclass에서 Function을 상속받는 예

```
class Person(object):  
    def speak(self):  
        print('I can speak')  
  
class Man(Person):  
    def wear(self):  
        print('I wear shirt')  
  
class Woman(Person):  
    def wear(self):  
        print('I wear skirt')  
  
man = Man()  
print( man.wear() )    # I wear shirt  
print( man.speak() )  # I can speak
```



****권고사항:** class의 이름의 첫번째글자는 대문자로 하는게 관행

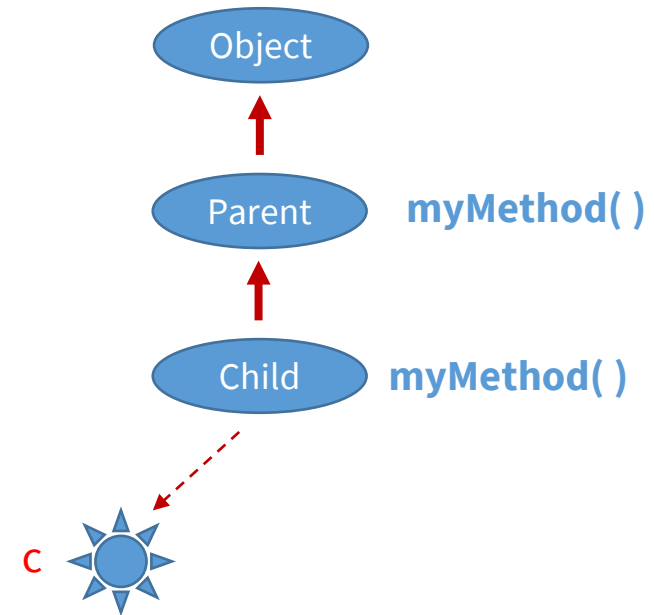
상속을 피하는 상황

- Subclass는 동일한 이름의 속성과 메서드를 직접 정의함으로써 부모 클래스의 속성과 메서드를 오버라이드(재정의) 가능

```
>>> class Parent:                # define parent class
    def myMethod(self):
        print('Calling the parent method')

>>> class Child(Parent):          # define child class
    def myMethod(self):
        print('Calling the child method')

>>> c = Child()                  # instance of child
>>> c.myMethod()                 # child calls overridden method
Calling the child method
```



Instance Variable vs. Class Variable

Instance Method vs. Class Method

- class SNU_Student

- Instance Variable

- 각각 Instance마다 다른값을 가지는 variable
- name, student_ID, courses, GPA

- Class Variable

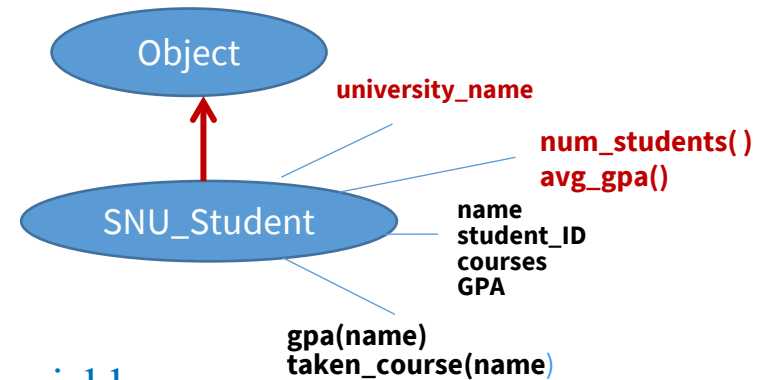
- Class의 모든 Instance에 공통적인값을 가지는 variable
- university_name

- Instance Method

- 각각 Instance에서 적용되는 method
- 학생이름을 input으로 학점을 return하는 method → gpa(name)
- 학생이름을 input으로 이수한 과목들을 return하는 method → taken_course(name)

- Class Method

- Class 내부의 전체 instance에 적용되는 method
- SNU_Student class에 있는 전체학생수를 return하는 method → num_students()
- SNU_Student class에 있는 전체학생수들인 평균GPA를 return하는 method → avg_gpa()

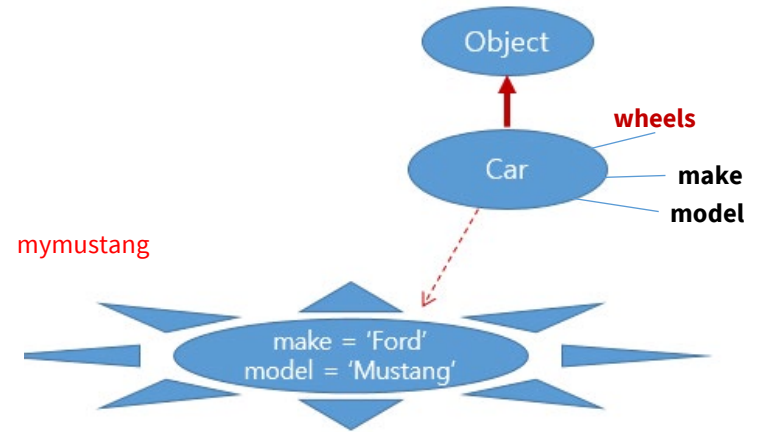


Accessing Class Variables from Both Instance and Class Name

```
class Car(object):  
    wheels = 4  
  
    def __init__(self, make, model):  
        self.make = make  
        self.model = model  
  
mymustang = Car('Ford', 'Mustang')  
  
print( mymustang.wheels )    # 4  
print( Car.wheels )         # 4
```

Class variable

Instance variable

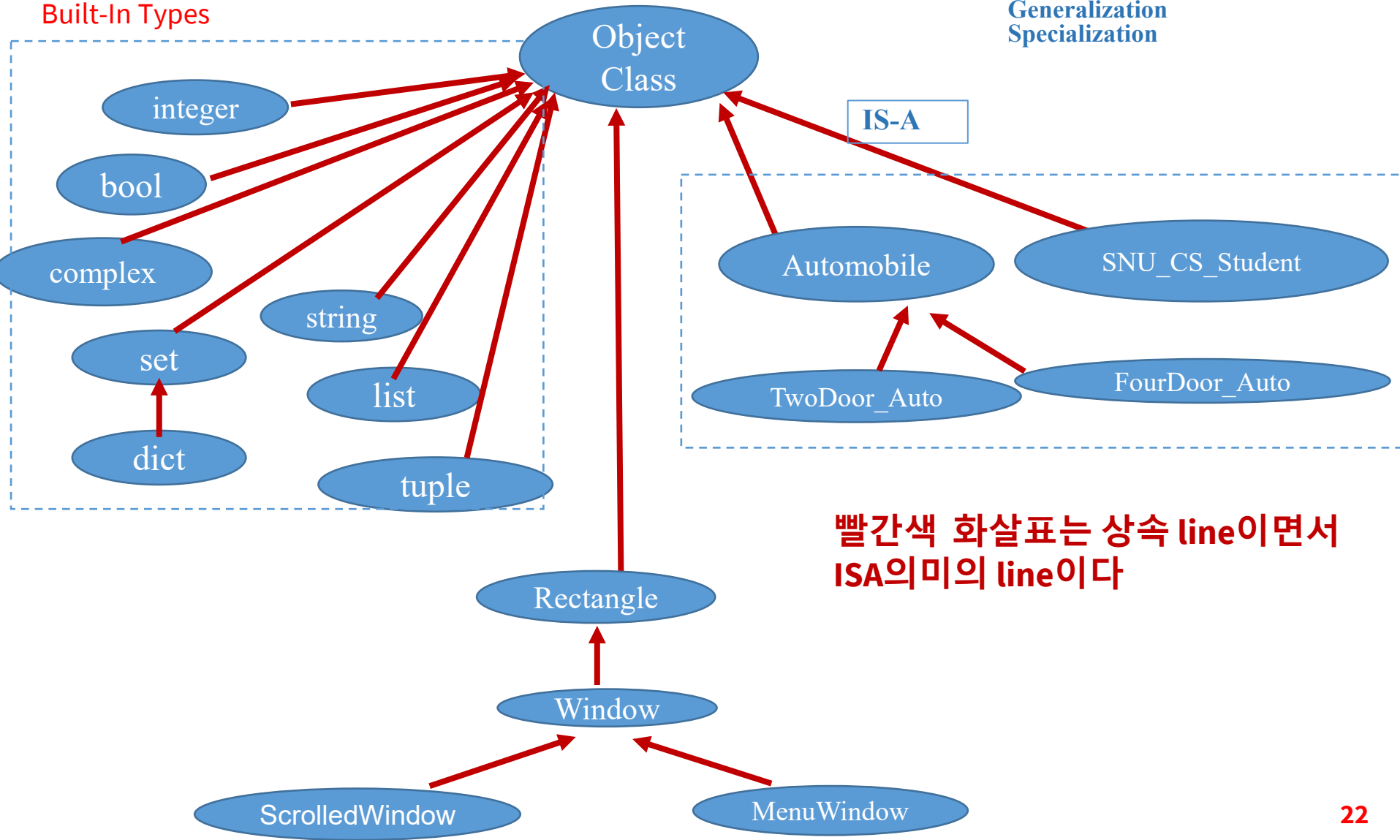


Everything is an Object in OOP

Python의 type system (class structure)

Built-In Types

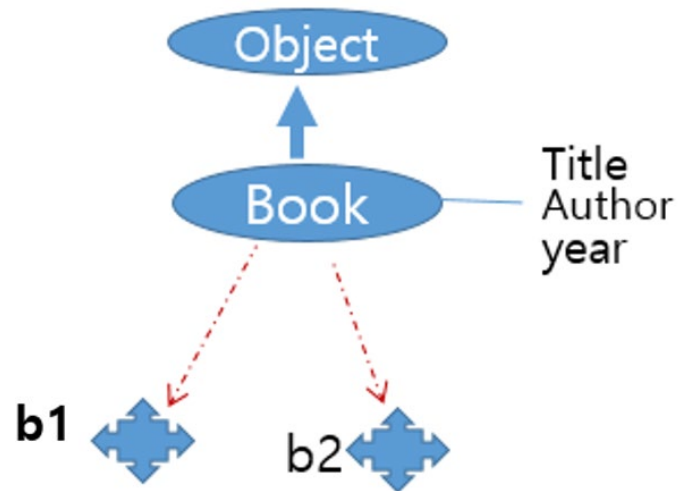
Generalization
Specialization



실전 Python OOP 예제들

도서관의 책들의 정보를 관리하고자 한다 [1/3]

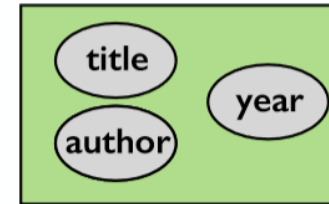
Book이라는 새로운 Data Type (class)를 만든다



Book이라는 새로운 Data Type (class)를 만든다 [2/3]

```
class Book(object):  
    def __init__(self):  
        self.title = None  
        self.author = None  
        self.year = None
```

Book class

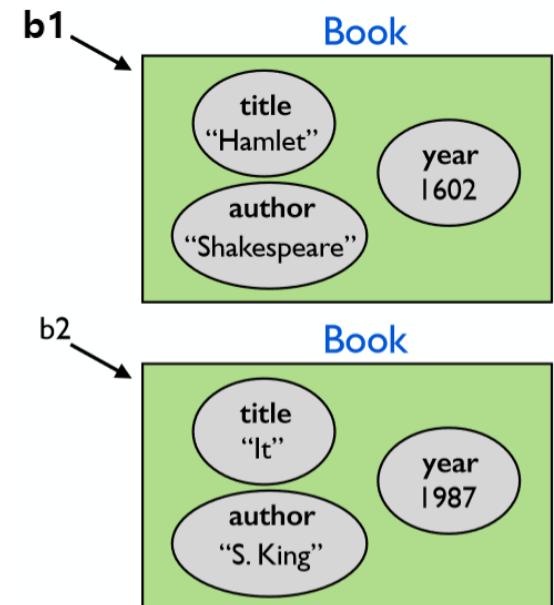


```
b1 = Book()  
b1.title = "Hamlet"  
b1.author = "Shakespeare"  
b1.year = 1602
```

b1 refers to an **object** of type **Book**.

```
b2 = Book()  
b2.title = "It"  
b2.author = "S. King"  
b2.year = 1987
```

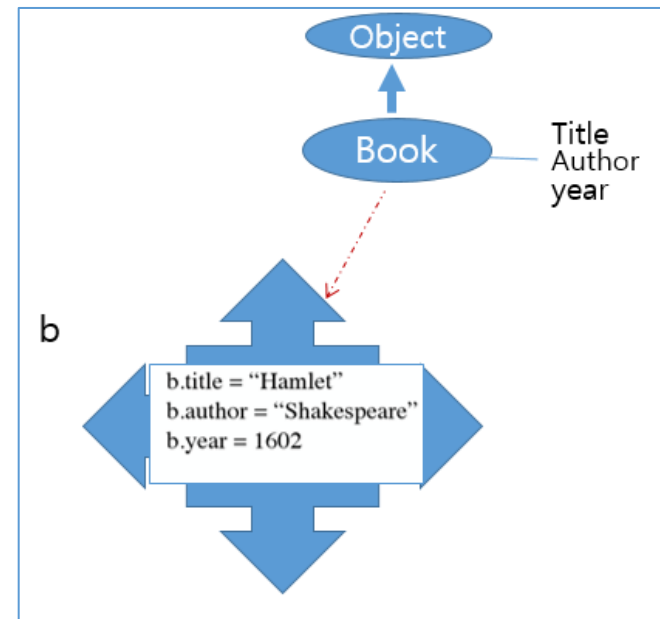
b2 refers to another **object** of type **Book**.



Book이라는 새로운 Data Type (class)를 만든다 [3/3]

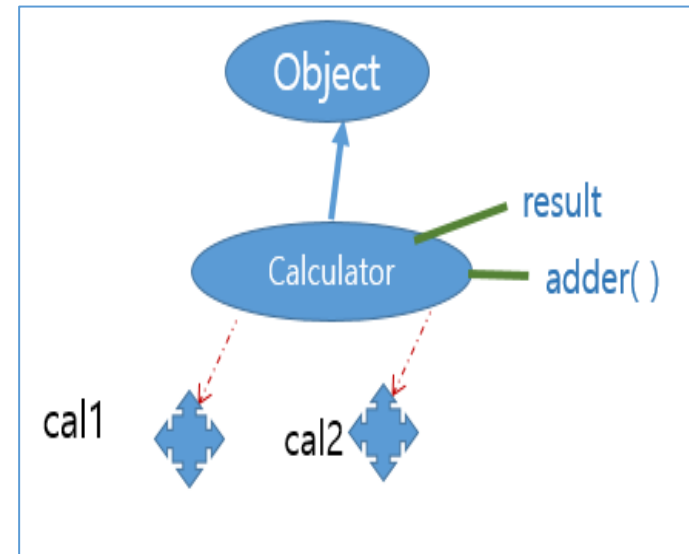
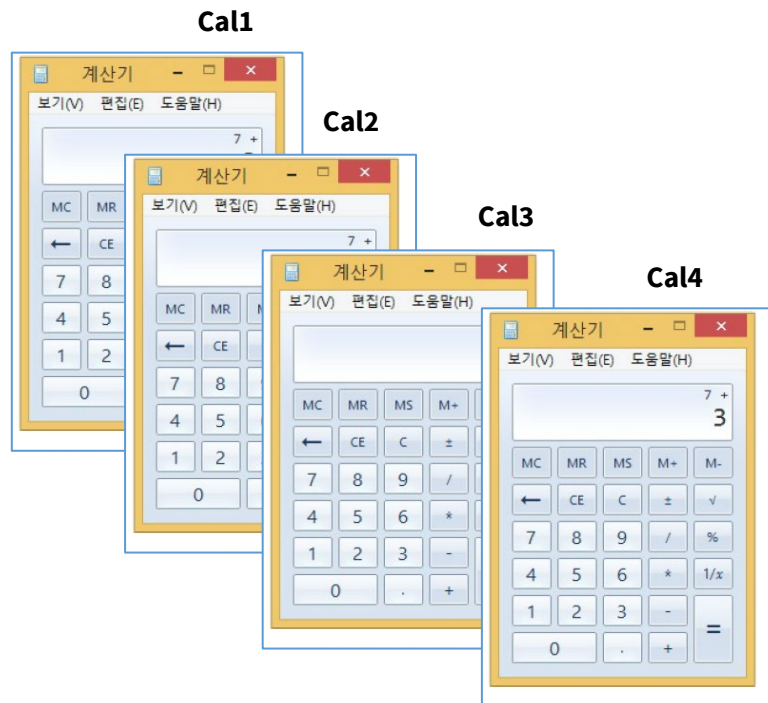
```
class Book(object):  
    def __init__(self, title, author, year):  
        self.title = title  
        self.author = author  
        self.year = year
```


b = Book("Hamlet", "Shakespeare", 1602)



계산기를 여러 개 만들어서 사용하고자 한다 [1/3]

Calculator이라는 새로운 Data Type (class)를 만든다



Calculator이라는 새로운 Data Type (class)를 만든다 [2/3]

Using Classes and Objects

```
class Calculator:
    def __init__(self):
        self.result = 0

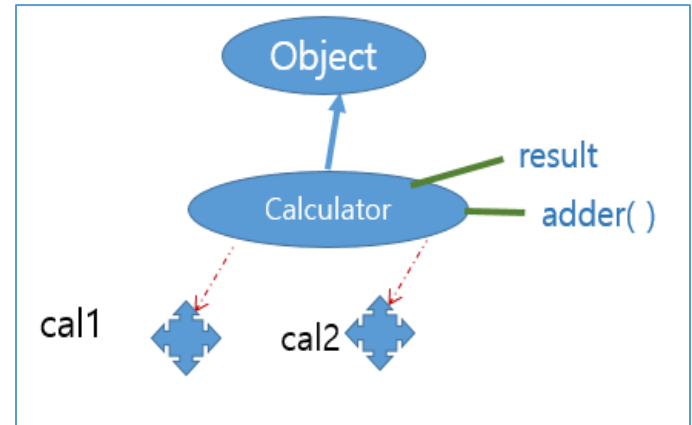
    def adder(self, num):
        self.result = self.result + num
        return self.result
```

←

```
cal1 = Calculator()
cal2 = Calculator()
```

```
print(cal1.adder(3))
print(cal1.adder(4))
print(cal2.adder(3))
print(cal2.adder(7))
```

```
3
7
3
10
```

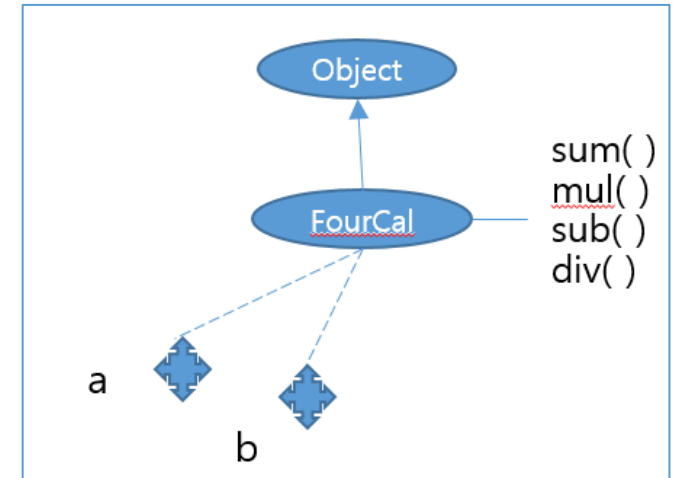


Calculator이라는 새로운 Data Type (class)를 만든다 [3/3]

Using Classes and Objects

```
class FourCal:
    def __init__(self, first, second):
        self.first = first
        self.second = second

    def sum(self):
        result = self.first + self.second
        return result
    def mul(self):
        result = self.first * self.second
        return result
    def sub(self):
        result = self.first - self.second
        return result
    def div(self):
        result = self.first / self.second
        return result
```



```
a = FourCal(4, 2)
b = FourCal(3, 7)
```

```
a.sum() # 6
a.mul() # 8
a.sub() # 2
a.div() # 2
```

```
b.sum() # 10
b.mul() # 21
b.sub() # -4
b.div() # 0
```

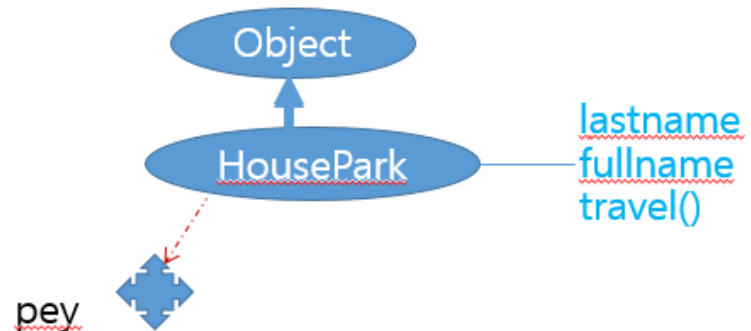
본격적인 실전 OOP 예제: 박씨네와 김씨네 [1/6]

- 아래의 상황을 Python OOP Coding 하고자 한다
 - 박씨집안 (HousePark) 사람들은 성이 박(Park) 이고 김씨집안 (HouseKim) 사람들은 성이 김(Kim) 이다
 - 박씨집안 (HousePark) 과 김씨집안 (HouseKim) 은 여행을 좋아하는 공통점이 있다.
 - 박씨집안 사람들은 여행을 갈때에 어디 장소로 여행간다는 내용을 공표한다
 - 김씨집안 사람들은 여행을 갈때에 어디 장소로 얼마기간 동안 여행간다는 내용을 공표한다
 - 어느날 두집안의 박응용군과 김줄리엣양은 사귀다가 결혼을 했다
 - 그러나 나중에 성격차로 싸움하고는 이혼을 했다

본격적인 실전 OOP 예제: 박씨네와 김씨네 [2/6]

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))

>>> pey = HousePark()
TypeError: __init__() takes exactly 2 arguments (1 given)
>>>
>>> pey = HousePark("응용")
>>> pey.travel("태국")
박응용, 태국여행을 가다.
```



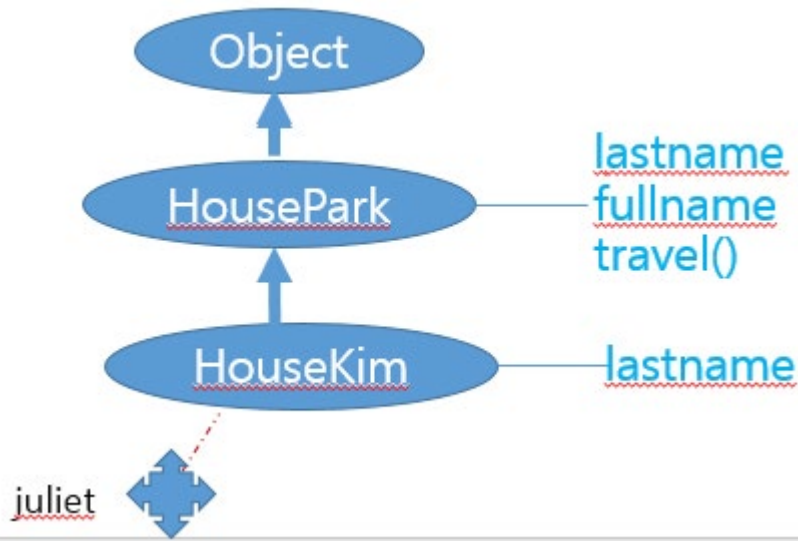
본격적인 실전 OOP 예제: 박씨네와 김씨네 [3/6]

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))

class HouseKim(HousePark):
    lastname = "김"
```

```
>>> juliet = HouseKim("줄리엣")
>>> juliet.travel("독도")
```

김줄리엣, 독도여행을 가다.



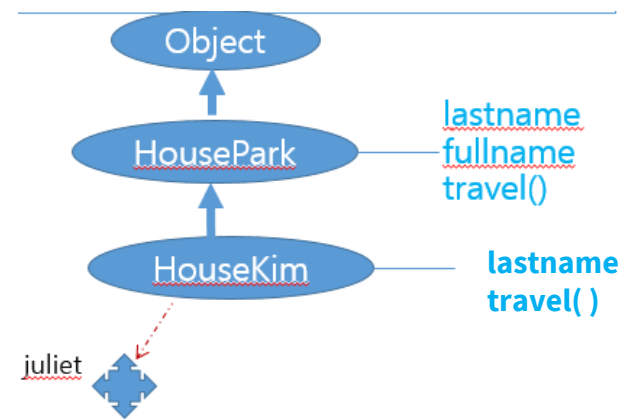
본격적인 실전 OOP 예제: 박씨네와 김씨네 [4/6]

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))

class HouseKim(HousePark):
    lastname = "김"
    def travel(self, where, day):
        print("%s, %s여행 %d일 가네." % (self.fullname, where, day))
```

```
>>> juliet = HouseKim("줄리엣")
>>> juliet.travel("독도", 3)
```

김줄리엣, 독도여행 3일 가네.



본격적인 실전 OOP 예제: 박씨네와 김씨네 [5/6]

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))
    def love(self, other):
        print("%s, %s 사랑에 빠졌네" % (self.fullname, other.fullname))
    def __add__(self, other):
        print("%s, %s 결혼했네" % (self.fullname, other.fullname))
```

Other가 가지고 있는 Object에
fullname 변수가 있다

```
class HouseKim(HousePark):
    lastname = "김"
    def travel(self, where, day):
        print("%s, %s여행 %d일 가네." % (self.fullname, where, day))
```

```
pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey.love(juliet)
pey + juliet
```

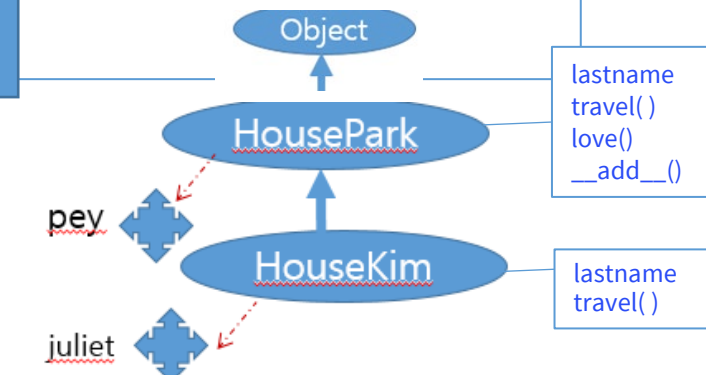
User-defined class의 instance에
+를 쓰려면 class내부에
__add__()가 define 되어있어야 함

박응용, 김줄리엣 사랑에 빠졌네
박응용, 김줄리엣 결혼했네

Remember?

[2, 4] + [3] → [2, 4, 3]

"S" + "N" + "U" → "SNU"



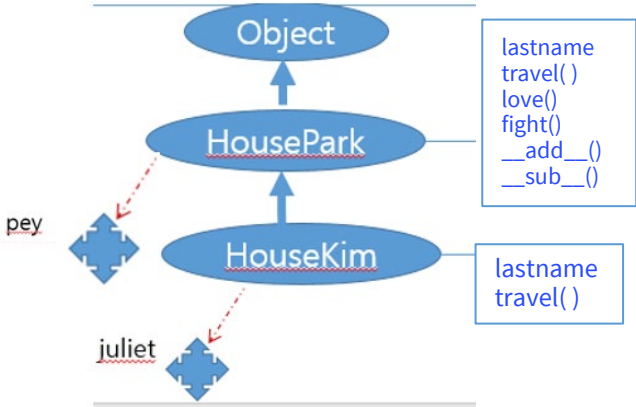
본격적인 실전 OOP 예제: 박씨네와 김씨네 [6/6]

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))
    def love(self, other):
        print("%s, %s 사랑에 빠졌네" % (self.fullname, other.fullname))
    def fight(self, other):
        print("%s, %s 싸우네" % (self.fullname, other.fullname))
    def __add__(self, other):
        print("%s, %s 결혼했네" % (self.fullname, other.fullname))
    def __sub__(self, other):
        print("%s, %s 이혼했네" % (self.fullname, other.fullname))

class HouseKim(HousePark):
    lastname = "김"
    def travel(self, where, day):
        print("%s, %s여행 %d일 가네." % (self.fullname, where, day))
```

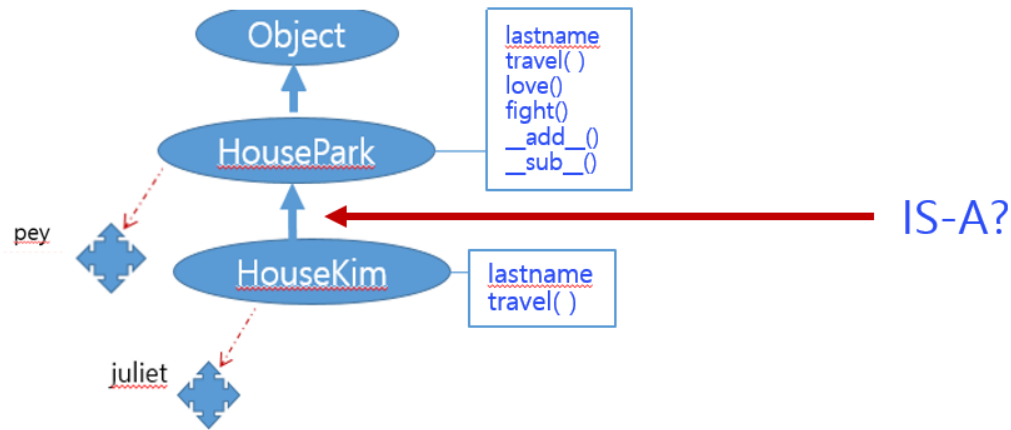
```
pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey.travel("부산")
juliet.travel("부산", 3)
pey.love(juliet)
pey + juliet
pey.fight(juliet)
pey - juliet
```

User-defined class의 instance에
- 를 instance에 쓰려면 __sub__()가
define 되어야 함



박응용 부산여행을 가다.
김줄리엣 부산여행 3일 가네.
박응용, 김줄리엣 사랑에 빠졌네
박응용, 김줄리엣 결혼했네
박응용, 김줄리엣 싸우네
박응용, 김줄리엣 이혼했네

박씨네와 김씨네 예제에서 여러가지 주요 OOP개념을 경험했으나!... ㅎㅎ



HousePark과 HouseKim은 Code Inheritance의 좋은 예제이나
IS-A 의미는 성립 안하는 부적절한 OOP예제

Special Class Method `__foo__()` 의 Mechanism

<code>__init__()</code>	for constructor
<code>__del__()</code>	for destructor
<code>__add__()</code>	for +
<code>__sub__()</code>	for -
<code>__mult__()</code>	for *
<code>__div__()</code>	for /
<code>__float__()</code>	for float function
<code>__str__()</code>	for print()
<code>__repr__()</code>	for creating computer readable form
<code>__hash__()</code>	for making hashable objects
<code>__lt__()</code>	for <
<code>__le__()</code>	for <=
<code>__gt__()</code>	for >
<code>__ge__()</code>	for >=
<code>__eq__()</code>	for ==
<code>__iter__()</code> , <code>__next__()</code>	for iterable object

- Class를 define할때에 상황에 필요한 `__foo__()`를 자체적으로 만드는것은 개발자의 몫
- Python에는 약 90개정도의 Special Class Method가 있다
- Magic Method, Double Underbar Method라고도 부른다